

# AI WEATHER WISE

## Introduction

Project Title : AI Weather Wise

Team ID : NM2026TMID38524

Team Leader : S. Srinidhi & [srinithi22nd@gmail.com](mailto:srinithi22nd@gmail.com)  
(CODER , VIDEO RECORDER)

Team Members :

R. Supriya & [supriyajagathi2006@gmail.com](mailto:supriyajagathi2006@gmail.com)  
(VOICE OVER)

R. Vasuki & [vasukiii1128@gmail.com](mailto:vasukiii1128@gmail.com)  
(DOCUMENTOR)

D.Yogeshwari & [priyadevaraj1307@gmail.com](mailto:priyadevaraj1307@gmail.com)  
(CODER)

## **Project overview**

AI WeatherWise is an intelligent weather information platform designed to provide users with real-time weather updates and AI-powered insights. The system combines weather data with Google Gemini AI to generate easy-to-understand summaries and personalized recommendations based on current environmental conditions. It also provides secure user authentication and features for managing favorite locations, making it a convenient and user-friendly platform for accessing weather information.

## **AI-Powered Weather Monitoring and Personalized Recommendations**

AI WeatherWise is designed for users who frequently travel to different cities and need a simple and intelligent way to monitor local weather conditions. Users often face difficulties tracking multiple locations, understanding raw weather data, and receiving personalized suggestions for activities, travel, and clothing. To address these challenges, AI WeatherWise provides a centralized platform that retrieves real-time weather information and uses Google Gemini AI to generate natural-language weather summaries and personalized recommendations. The system also includes secure user authentication and allows users to manage their favorite locations and preferences, providing a convenient and personalized weather experience.

By leveraging AI and centralized data management, users spend less time deciphering raw metrics. The system ensures data integrity via Mongoose schemas and provides high-value, context-aware weather advice through Gemini integrations.

## Software Requirements

- ❖ **Operating System:** Windows 10/11, macOS, or Linux.
- ❖ **Node.js:** Version 16 or above for backend development and server execution.
- ❖ **npm:** Version 8 or above for managing project packages and dependencies.
- ❖ **Google Gemini AI:** Used to generate AI-based weather summaries and personalized recommendations.
- ❖ **MongoDB:** Used to store user accounts, favorite locations, preferences, and weather-related data.
- ❖ **Postman:** Used for testing and verifying REST APIs.
- ❖ **Visual Studio Code:** Used as the primary code editor for development.
- ❖ **Web Browser:** Google Chrome, Microsoft Edge, or Mozilla Firefox for accessing the application.

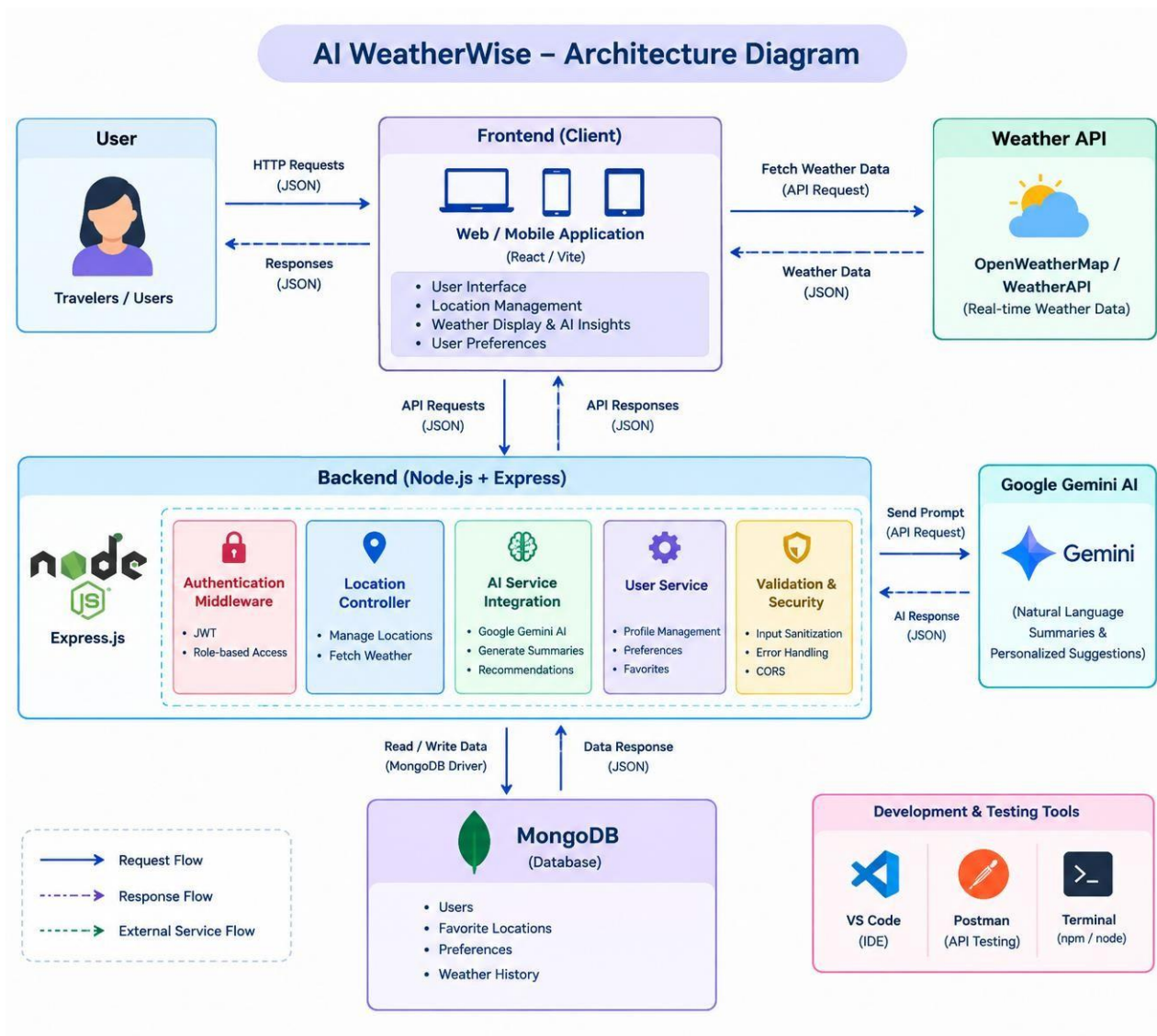
## Hardware Requirements

- ❖ **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent.
- ❖ **RAM:** 8 GB minimum; 16 GB recommended.
- ❖ **Storage:** At least 1 GB of free space.
- ❖ **Internet Connection:** Required for accessing real-time weather data, MongoDB Atlas, and Gemini AI services.

## Key Features

1. **User Authentication:** Secure registration and login using JWT-based authentication.
2. **Real-Time Weather Updates:** Fetches current weather information for selected cities and locations.
3. **Favorite Locations:** Allows users to save and manage their frequently visited cities.
4. **AI Weather Summaries:** Uses Google Gemini AI to provide easy-to-understand weather descriptions.
5. **Personalized Recommendations:** Suggests suitable activities, clothing, and travel plans based on weather conditions.
6. **User Preferences:** Stores user preferences and settings for a personalized experience.
7. **Secure Data Management:** Protects user information and application data using authentication and validation mechanisms.

# Architecture



## Project Configuration

### Creating the Project Folder

- Create a new folder named **AI Weather wise**
- Open the project folder in **Visual Studio Code**.
- Initialize the project using Node.js and install the required dependencies.

## Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init-y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer@google/genai
```

Install Nodemon for development:

```
npm install --save-dev nodemon
```

## Create the Following Files

- ❖ index.js
- ❖ .env
- ❖ package.json
- ❖ .gitignore

## Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

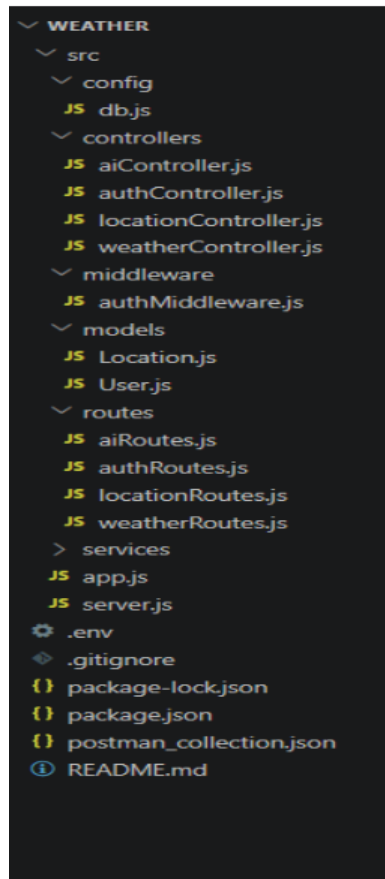
```
JWT_SECRET=your_jwt_secret
```

```
GEMINI_API_KEY=your_gemini_api_key
```

## Backend Development

The AI Weatherwise backend is built with **Node.js**, **Express.js**, and **MongoDB** using a structured and modular design. Different components such as **routes**, **controllers**, **middleware**, **models**, and **configuration files** are organized separately to make the system easier to maintain and expand. It provides secure **RESTful APIs** for user authentication,

weather information, location management, and AI-based recommendations. **JWT** is used for secure user authentication, while **MongoDB and Mongoose** handle data storage and management. The system also uses **Google Gemini AI** to provide personalized weather insights, including activity and clothing suggestions based on current weather conditions.



## Core System Script Implementations

### 1.Application Initialization (index.js)

The **index.js** file acts as the main starting point of the application. It initializes the Express server, loads the environment variables, configures the required middleware, establishes a connection with the MongoDB database, registers the API routes, and starts the server once the database connection is successfully established.

```
const app = require('./src/app');
const connectDB = require('./src/config/db');

require('dotenv').config();

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      console.log('Press Ctrl+C to terminate.');
```

## Database Configuration

### API Testing Demo

#### 1. MongoDB Setup

##### ❖ Install Mongoose

Install the Mongoose package in the Node.js project using the following command:

```
npm install mongoose
```

### ❖ Establish Database Connection

Set up the MongoDB connection in the `config/db.js` file using the MongoDB URI specified in the `.env` configuration file.

### ❖ Define Schemas and Models

Create Mongoose schemas and models to organize and manage application data, including user information, weather details, and saved locations.

### ❖ Verify Database Connectivity

Start the Node.js server and check whether the application connects successfully to MongoDB.

## Mongoose Data Models

Mongoose data models define the structure and data types of documents stored in MongoDB. They ensure data consistency and make it easier for the application to perform database operations such as creating, retrieving, updating, and deleting records.

```
const mongoose = require('mongoose');

const locationSchema = new mongoose.Schema({
  city: {
    type: String,
    required: [true, 'Please provide a city name'],
    trim: true
  },
  country: {
    type: String,
    required: [true, 'Please provide a country name'],
    trim: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: [true, 'User reference is required']
  }
}, {
  timestamps: true
});

// Ensure a user cannot duplicate the exact same city and country in their favorites
locationSchema.index({ user: 1, city: 1, country: 1 }, { unique: true });

const Location = mongoose.model('Location', locationSchema);
module.exports = Location;
```

# User Schema

Defines the structure of user information stored in MongoDB, such as name, email, password, and other account details. It helps maintain consistent and organized user data in the application.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*(\\.\\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

# Db connection

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);

    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

## Project Execution

The AI Weatherwise backend is run locally using Node.js. Before launching the application, make sure MongoDB is properly connected and all required environment variables are set in the .env file. The available REST APIs can be verified and tested using Postman or Thunder Client.

### Starting the Backend Server

Install the required packages:

```
npm install
```

Run the application:

```
npm start
```

After the server starts successfully, it connects to the MongoDB database and becomes ready to handle API requests.

# API testing:

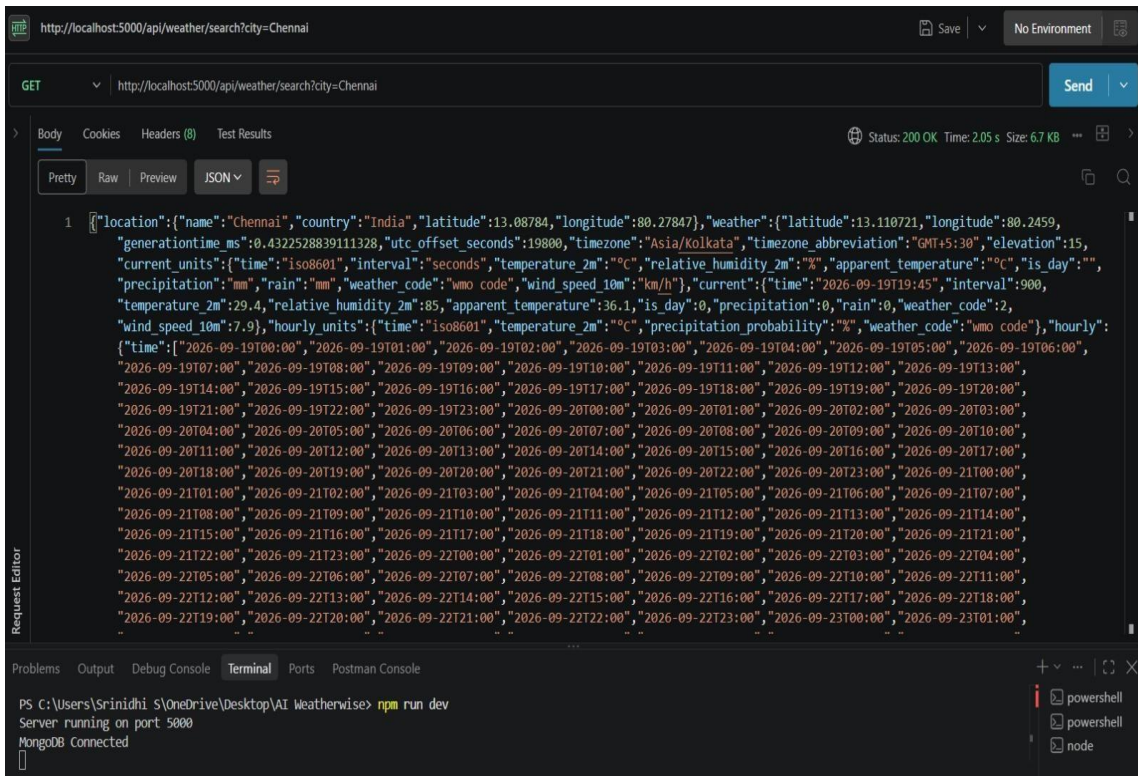
## Register API:

The screenshot shows a Postman interface for a POST request to `http://localhost:5000/api/auth/register`. The request body is a JSON object: `{ "name": "Srinidhi", "email": "srinidhi@gmail.com", "password": "12345678" }`. The response status is 201 Created, with a time of 270 ms and a size of 403 B. The response body is: `{ "message": "User registered successfully", "user": { "id": "6aae95dc43ce222352620762", "name": "Srinidhi", "email": "srinidhi@gmail.com" } }`. The terminal at the bottom shows the command `npm run dev` being executed, with output indicating the server is running on port 5000 and MongoDB is connected.

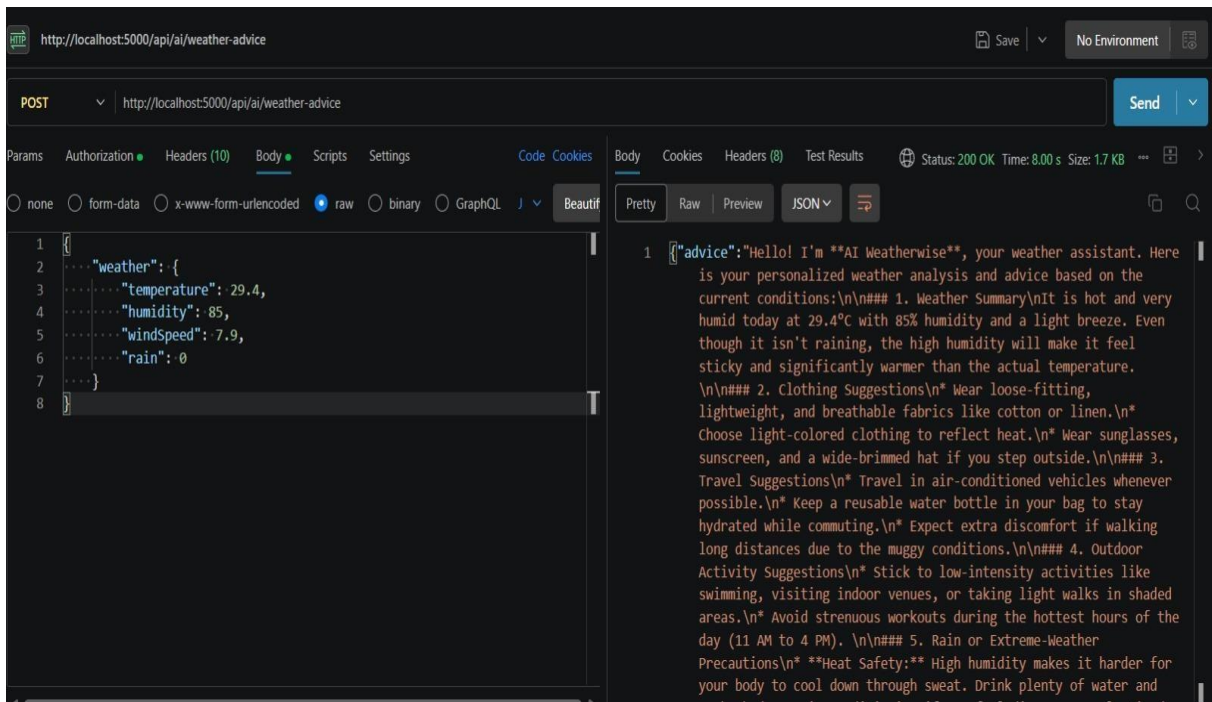
## Login API:

The screenshot shows a Postman interface for a POST request to `http://localhost:5000/api/auth/login`. The request body is a JSON object: `{ "email": "srinidhi@gmail.com", "password": "12345678" }`. The response status is 200 OK, with a time of 224 ms and a size of 574 B. The response body is: `{ "message": "Login successful", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmVzI2YyZyZyYmJlZNTI2MjA3NjIiLCJpYXQiOiE3ODk4Mjg1NzEsImV4cCI6MTc5MDQzMzY3MzY0Lj0tbnU65qcl40bMY7mqbFcsF4zdmYd8WahAX1yJWac26Y", "user": { "id": "6aae95dc43ce222352620762", "name": "Srinidhi", "email": "srinidhi@gmail.com" } }`. The terminal at the bottom shows the command `npm run dev` being executed, with output indicating the server is running on port 5000 and MongoDB is connected.

# Weather API:



# Recommendation API:



**API testing.mp4**

**<https://drive.google.com/file/d/1pf7PZ8s5S39lv0EyTzBeccOta2v-aqN7/view?usp=sharing>**

**Demo Video link:**

**[https://drive.google.com/file/d/1HnMDJOp8thpXhi57g\\_IuDocmZ0YBpaM5/view?usp=sharing](https://drive.google.com/file/d/1HnMDJOp8thpXhi57g_IuDocmZ0YBpaM5/view?usp=sharing)**